

MIST: Meta-Learning Latent State Transitions for Implicit Recovery in Multi-Drone Payload Transport

Harshit Moondra

Abstract—Mid-episode actuator faults in cooperative UAV payload transport alter the team’s effective force distribution in a way that is indistinguishable from nominal dynamics in the observation space, preventing adaptation by standard multi-agent policies. We present MIST, a framework that augments a shared MAPPO policy with a GRU-based context encoder that infers per-drone actuator effectiveness from trajectory history, coupled with a latent warmup schedule that resolves the distribution mismatch between supervised encoder pre-training and on-policy rollouts. Evaluated across seven mid-episode fault configurations in a PyBullet quadrotor simulation, MIST matches a vanilla MAPPO baseline on mean reward while substantially reducing per-scenario variance, and both on-policy methods outperform a MADDPG baseline on payload positioning accuracy. Ablation confirms that the warmup schedule is critical: without it, the policy fails to incorporate encoder context and performance degrades severely.

I. INTRODUCTION AND MOTIVATION

Cooperative payload transport using teams of unmanned aerial vehicles (UAVs) offers significant advantages over single-robot systems: distributed force generation allows heavier payloads, redundant agents improve robustness, and flexible team composition enables reconfiguration in the field [1], [2]. At the same time, the strong dynamical coupling introduced by a shared payload means that the behavior of one agent directly affects every other — a property that makes the system both powerful and fragile. In particular, a mid-flight actuator fault on any single drone propagates through the team interaction and immediately destabilizes the payload, often before a nominal controller can react [3], [4].

Standard multi-agent reinforcement learning (MARL) policies trained under nominal conditions [5] have no mechanism for detecting or adapting to such faults at runtime. The policy observes the same state space before and after a failure and has no way to distinguish degraded team dynamics from wind or other disturbances. Conditioning the policy on a learned representation of the fault state offers a principled solution: a context encoder trained with supervised fault labels maps recent trajectory history to a compact latent variable z encoding the per-drone health configuration, so that a single policy conditioned on z can adapt to any fault scenario without retraining. This draws conceptual inspiration from context-conditioned meta-RL approaches [6], but differs in a key respect: rather than inferring a posterior over z via variational inference, we train a deterministic GRU encoder with supervised decode heads, trading probabilistic calibration for the computational simplicity required by on-policy training.

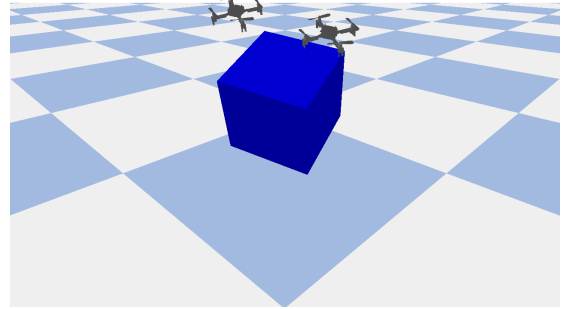


Fig. 1: PyBullet simulation environment. Two Crazyflie 2.X quadrotors (CF2X) are rigidly attached to a blue cubic payload and cooperatively hold it at the target hover height of 0.5 m. Faults are injected at step 100 of each episode by modifying the actuator effectiveness of one or both drones.

A. Simulation Environment

This work uses the `gym-pybullet-drones` simulator [7], which provides a high-fidelity PyBullet physics environment for Crazyflie 2.X quadrotors (Fig. 1). Two CF2X drones are rigidly attached to a lightweight payload ($M_p = 0.02$ kg) via fixed constraints, positioned symmetrically at a radius of 0.10 m from the payload center of mass. The drones are controlled at 240 Hz using direct rotor RPM commands (4-dimensional action space per drone), and observations consist of 12-dimensional kinematic states (position, orientation, velocity, angular velocity) augmented with 12-dimensional payload state (position, orientation, velocity, angular velocity). The transport task is to maintain the payload at a fixed target position $p^* = [0, 0, 0.5]^\top$ m for a 200-step episode.

Faults are injected at step 100 of each episode by setting a drone’s health state to one of three modes:

- **Nominal** ($b_i = 0$): full actuator effectiveness ($\alpha_i = 1.0$)
- **Degraded** ($b_i = 1$): partial effectiveness ($\alpha_i \sim \mathcal{U}[0.3, 0.7]$)
- **Failed** ($b_i = 2$): zero thrust ($\alpha_i = 0.0$)

This yields seven distinct two-drone fault configurations, from fully nominal $[0, 0]$ through worst-case failed-plus-degraded $[2, 1]$, covering every meaningful combination of health states.

B. Contributions

Building on the problem formulation and MIST architecture proposed at the midterm, this paper delivers the following:

- 1) A **GRU-based context encoder** that maps a window of recent observations, actions, and residual wrenches to a 32-dimensional latent vector encoding each drone's health state and actuator effectiveness, trained with a composite loss combining health classification, effectiveness regression, and residual wrench reconstruction.
- 2) Integration of the context encoder with **MAPPO** (Multi-Agent Proximal Policy Optimization) using a context-augmented actor and centralized critic, with separate actor and critic optimizers to prevent value-loss gradients from dominating policy learning.
- 3) A **latent warmup schedule** that scales the encoder's contribution from zero to full over the first 60k training steps ($\beta = 0.15$ of 4×10^5 total), resolving the distribution mismatch between encoder pre-training data (PD controller trajectories) and on-policy rollouts.
- 4) A **full empirical comparison** against vanilla MAPPO and a MADDPG baseline across all seven fault scenarios, showing that MIST matches vanilla MAPPO on mean reward (-55.0 vs. -54.3) with substantially lower variance (std 0.3 – 1.8 vs. 0.7 – 7.5), and that both on-policy methods outperform MADDPG on payload drift (0.383 m vs. 0.521 m).

II. PROBLEM FORMULATION

A. Physical System

Consider a rigid payload of mass M_p and inertia tensor $\mathbf{I}_p$ cooperatively transported by $N = 2$ UAVs [8], [1]. The payload configuration is $Y = (p_p, R_p) \in SE(3)$, where $p_p \in \mathbb{R}^3$ is the center-of-mass position and $R_p \in SO(3)$ is the orientation. The payload velocity is $\nu = [v_p^\top, \omega_p^\top]^\top \in \mathbb{R}^6$. Each UAV is rigidly attached to the payload at a fixed offset $r_i \in \mathbb{R}^3$ expressed in the payload frame, so the UAV positions are fully determined by Y .

All vectors are expressed in the world (inertial) frame. Let $r_i = R_p r_i^b \in \mathbb{R}^3$ be the lever arm from the payload center of mass to UAV i in the world frame, where r_i^b is the fixed attachment offset in the payload body frame and $R_p \in SO(3)$ is the payload orientation. Let $f_i \in \mathbb{R}^3$ be the thrust of UAV i expressed in the world frame. The Newton–Euler payload dynamics are then

$$M_p \dot{\nu}_p = \sum_{i=1}^N f_i - M_p g e_3, \quad (1)$$

$$\mathbf{I}_p \dot{\omega}_p = \sum_{i=1}^N r_i \times f_i - \omega_p \times (\mathbf{I}_p \omega_p), \quad (2)$$

where $g = 9.81 \text{ ms}^{-2}$ and $e_3 = [0, 0, 1]^\top$. Defining the applied wrench as $\mathcal{W} = [\sum_i f_i^\top, \sum_i (r_i \times f_i)^\top]^\top \in \mathbb{R}^6$, and stacking all forces into $u = [f_1^\top, \dots, f_N^\top]^\top \in \mathbb{R}^{3N}$, we have $\mathcal{W} = \mathbf{G}u$ where the grasp matrix is

$$\mathbf{G} = \begin{bmatrix} \mathbf{I}_3 & \cdots & \mathbf{I}_3 \\ \hat{r}_1 & \cdots & \hat{r}_N \end{bmatrix} \in \mathbb{R}^{6 \times 3N}, \quad (3)$$

and $\hat{r}_i \in \mathbb{R}^{3 \times 3}$ is the skew-symmetric matrix of r_i . For the symmetric two-drone configuration with $r_1 = -r_2$ and $\|r_i\| = 0.10$ m, $\mathbf{G} \in \mathbb{R}^{6 \times 6}$ is full rank, meaning the map

from joint forces to applied wrench is surjective over $\mathbb{R}^6$; in practice the achievable wrench set is a bounded convex polytope determined by each rotor's thrust limits [9].

B. Fault Model

Each drone i has a latent health state $b_i \in \{0, 1, 2\}$ and a scalar effectiveness coefficient $\alpha_i \in [0, 1]$, defined as

$$\alpha_i = \begin{cases} 1.0, & b_i = 0 \text{ (nominal)}, \\ \mathcal{U}[0.3, 0.7], & b_i = 1 \text{ (degraded)}, \\ 0.0, & b_i = 2 \text{ (failed)}. \end{cases} \quad (4)$$

Each rotor produces thrust proportional to the square of its angular speed; scaling all four rotors of drone i by $\sqrt{\alpha_i}$ therefore scales the net body-frame thrust by α_i , so the effective force is $\tilde{f}_i = \alpha_i f_i$ [3]. Faults are injected at step $t_f = 100$ of each 200-step episode; health states are held fixed for the remainder. The team health vector $b = [b_1, \dots, b_N]^\top$ is *not* observable to any agent, making the health state a hidden variable that must be inferred from trajectory data.

The fault-conditioned grasp matrix is

$$\mathbf{G}(b) = \begin{bmatrix} \alpha_1 \mathbf{I}_3 & \cdots & \alpha_N \mathbf{I}_3 \\ \alpha_1 \hat{r}_1 & \cdots & \alpha_N \hat{r}_N \end{bmatrix}, \quad (5)$$

which has rank less than 6 when any $\alpha_i = 0$, strictly reducing the Attainable Wrench Set [9] and making full payload stabilization impossible without compensation from the remaining drone.

C. Multi-Agent Markov Decision Process

The cooperative transport task is modeled as a Dec-POMDP [10], [11], a tuple $\langle N, \mathcal{S}, \{\mathcal{A}^i\}, T, R, \{\mathcal{O}^i\}, \Omega, \gamma \rangle$ defined as follows.

State space. The full hidden state $x_t = (Y_t, \nu_t, b) \in \mathcal{S}$ comprises the payload configuration $Y_t \in SE(3)$, payload velocity $\nu_t \in \mathbb{R}^6$, and the hidden health vector $b \in \{0, 1, 2\}^N$. Because rigid attachment constrains each drone's position and velocity to $(p_i, v_i) = (p_p + R_p r_i^b, \dot{p}_p + \omega_p \times R_p r_i^b)$, the per-drone kinematic terms in the observation are deterministic functions of (Y_t, ν_t) and do not introduce additional state dimensions; they are included in o_t^i to provide a richer local signal. Because b is absent from the observable part of x_t , agents cannot distinguish states with identical kinematics but different health configurations, making this a genuine POMDP rather than an MDP [5].

Transition. The kinematic part of T follows the fault-conditioned Newton-Euler dynamics (1)–(2) with $\tilde{f}_i = \alpha_i f_i$, simulated at 240 Hz via PyBullet. The health vector b is drawn once per episode from the uniform distribution over all $3^N = 9$ configurations in $\{0, 1, 2\}^N$ and injected deterministically at step $t_f = 100$; it remains fixed for the episode. Evaluation reports seven of these nine configurations. Two are excluded: $[2, 2]$ (both drones failed), because complete thrust loss makes all policies equivalent and provides no meaningful differentiation; and $[1, 2]$ (drone-0 degraded, drone-1 failed), because the two Crazyflie drones are mechanically homogeneous, making $[1, 2]$ a relabelling of the evaluated $[2, 1]$ scenario.

Observation. Each agent i receives a local observation from the function $\Omega^i : \mathcal{S} \rightarrow \mathcal{O}^i$,

$$o_t^i = [p_i, \text{rpy}_i, v_i, \omega_i, p_p, \text{rpy}_p, v_p, \omega_p] \in \mathbb{R}^{24}, \quad (6)$$

comprising the 12-dimensional kinematic state of drone i concatenated with the 12-dimensional payload state; b is absent from o_t^i by construction.

Global state. Under the centralized-training / decentralized-execution (CTDE) paradigm [12], the critic accesses the joint observed state $\bar{s}_t = [o_t^1; \dots; o_t^N] \in \mathbb{R}^{48}$ during training only; $\bar{s}_t$ is observable (unlike x_t) since it contains no health information beyond what the observations provide.

Action. Each agent outputs $a_t^i \in [-1, 1]^4$, a normalized four-dimensional RPM command that is linearly mapped to the operating range of the Crazyflie 2.X rotors.

Reward. All agents share a common team reward

$$r_t = -\|p_p - p^*\| - 0.01 \|\omega_p\| - 0.5 |p_{p,z} - p_z^*|, \quad (7)$$

where $p^* = [0, 0, 0.5]^\top$ m is the target hover position. The three terms penalize Euclidean position error, payload tumbling (rotational velocity), and vertical deviation respectively; the asymmetric weight on the z -axis reflects the higher cost of payload drop versus lateral excursion.

Context-augmented policy. The MIST actor conditions on a latent fault context inferred from trajectory history:

$$\hat{o}_t^i = [o_t^i; \lambda \cdot z_t] \in \mathbb{R}^{24+d_z}, \quad (8)$$

where $z_t = g_\phi(\tau_t) \in \mathbb{R}^{32}$ is the GRU encoder output, τ_t is a sliding window of recent observations, actions, and residual wrenches, $\lambda \in [0, 1]$ is the latent warmup scale, and $d_z = 32$. Each agent runs the shared actor $\pi_\theta(a_t | \hat{o}_t^i)$, and the centralized critic evaluates $V_\psi(\hat{s}_t)$ where $\hat{s}_t = [\bar{s}_t; \lambda z_t]$.

Objective. The team seeks a policy that maximizes cumulative discounted return

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad (9)$$

under the distribution of fault configurations $b \sim p(b)$ and fault injection time $t_f = 100$, where $\gamma = 0.99$ and $T = 200$.

III. MAIN RESULTS

A. Context Encoder Pre-Training

The context encoder g_ϕ is a GRU-based recurrent network that maps a trajectory history window $\tau_t = (o_{t-H}, a_{t-H}, r_{t-H}, \dots, o_t, a_t, r_t)$ to a latent vector $z_t \in \mathbb{R}^{32}$, where $r_t = \mathcal{W}_{\text{obs},t} - \mathbf{G}u_t$ is the residual wrench at each step. From z_t two decode heads produce: a categorical health distribution $\hat{b}_i = h_\psi(z_t)$ and a scalar effectiveness estimate $\hat{\alpha}_i = e_\xi(z_t)$ for each drone. A third head reconstructs the last residual wrench, providing a self-supervised signal that grounds the latent in observable physics.

The encoder is pre-trained on trajectories collected by a PD baseline controller across all 3^N fault configurations (Algorithm 1). The composite training loss is

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{CE}}(\hat{b}, b)}_{\text{health class.}} + \underbrace{\|\hat{\alpha} - \alpha\|^2}_{\text{effectiveness}} + \underbrace{\|\hat{r} - r_T\|^2}_{\text{residual recon.}} \quad (10)$$

Algorithm 1 Context Encoder Pre-Training (MIST)

Require: Fault configs $\mathcal{B}$, PD controller π_{pd} , episode length T , epochs E , batch size B

Ensure: Trained encoder parameters ϕ, ψ, ξ

```

1: Initialise  $\phi, \psi, \xi$  randomly
2:  $\mathcal{D} \leftarrow \emptyset$ 
3: for each fault config  $b \in \mathcal{B}$  do ▷ data collection
4:   Roll out  $\pi_{\text{pd}}$  for  $T$  steps with fault  $b$  injected at  $t=T/2$ 
5:   Store  $(\tau, b, \alpha, r_{1:T})$  in  $\mathcal{D}$ 
6: end for
7: for epoch  $e = 1, \dots, E$  do
8:   Sample mini-batch  $\{(\tau^{(k)}, b^{(k)}, \alpha^{(k)}, r_T^{(k)})\}_{k=1}^B$  from  $\mathcal{D}$ 
9:    $z \leftarrow \text{GRU}_\phi(\tau)$  ▷ encode trajectory
10:   $\hat{b} \leftarrow h_\psi(z), \hat{\alpha} \leftarrow e_\xi(z), \hat{r} \leftarrow d_\phi(z)$ 
11:  Compute  $\mathcal{L}$  via (10)
12:  Update  $\phi, \psi, \xi$  by Adam gradient step on  $\mathcal{L}$ 
13: end for
```

where b and α are the ground-truth health states and effectiveness values provided by the simulator during pre-training, and r_T is the residual wrench at the final step of the window.

B. MAPPO with Context Encoder and Latent Warmup

After pre-training, the encoder is frozen and integrated with MAPPO [13]. MAPPO uses a shared Gaussian actor $\pi_\theta(a_t | \hat{o}_t)$ and a centralised value critic $V_\psi(\hat{s}_t)$, where $\hat{o}_t = [o_t; \lambda \cdot z_t]$ is the observation augmented with the scaled encoder latent and $\hat{s}_t = [\bar{s}_t; \lambda z_t]$ is the joint observed state similarly augmented.

A key challenge is that the encoder was pre-trained on PD-controller trajectories, while during MAPPO training the policy generates different, initially random rollouts. This distribution mismatch causes the encoder to emit noisy latents early in training, which degrades policy learning. We address this with a *latent warmup schedule*: the scale factor λ is annealed linearly from 0 to 1 over the first β fraction of training steps (we use $\beta = 0.15$, corresponding to 60k absolute warmup steps). This lets the actor first learn a stable base policy on observations alone, then gradually incorporate fault context as the rollout distribution stabilises. The full procedure is given in Algorithm 2.

C. MADDPG Baseline

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [12] serves as an off-policy baseline under the same centralized-training / decentralized-execution (CTDE) paradigm [13]. A single shared MLP actor $\mu_\theta : \mathbb{R}^{24} \rightarrow [-1, 1]^4$ maps each agent's local observation to a deterministic action. A centralized critic $Q_\omega(\bar{s}_t, a_t^1, \dots, a_t^N)$ conditions on the joint observed state $\bar{s}_t \in \mathbb{R}^{48}$ and all agents' actions, giving a single scalar value estimate; both actor and critic are weight-shared across the two homogeneous drones.

Training follows the standard DDPG recipe: transitions are stored in a replay buffer of capacity 10^5 ; gradients are computed on mini-batches of 128 transitions; target networks $\mu_{\theta'}$,

Algorithm 2 MAPPO with Context Encoder and Latent Warmup

Require: Frozen encoder g_ϕ , total steps S , rollout length L , PPO epochs K , clip ϵ , warmup fraction β , entropy schedule (η_0, η_f)

Ensure: Policy parameters θ , critic parameters ψ

```

1: Initialise  $\theta, \psi$  randomly;  $\lambda \leftarrow 0$ ;  $s \leftarrow 0$ 
2: while  $s < S$  do
3:    $\lambda \leftarrow \min(1, s/(\beta S))$   $\triangleright$  latent warmup
4:    $\eta \leftarrow \eta_0 + (s/S)(\eta_f - \eta_0)$   $\triangleright$  entropy annealing
5:   for step  $t = 1, \dots, L$  do  $\triangleright$  rollout collection
6:      $z_t \leftarrow g_\phi(\tau_t)$ ;  $\hat{o}_t \leftarrow [o_t; \lambda z_t]$ ;  $\hat{s}_t \leftarrow [\bar{s}_t; \lambda z_t]$ 
7:     Sample  $a_t \sim \pi_\theta(\cdot | \hat{o}_t)$ , execute, observe  $r_t, o_{t+1}$ 
8:     Store  $(\hat{o}_t, \hat{s}_t, a_t, \log \pi_\theta(a_t | \hat{o}_t), V_\psi(\hat{s}_t), r_t)$ 
9:   end for
10:  Compute returns and advantages via GAE [14]
11:  for PPO epoch  $k = 1, \dots, K$  do
12:    for each mini-batch  $\mathcal{M}$  do
13:       $\rho \leftarrow \pi_\theta(a | \hat{o}) / \pi_{\theta_{\text{old}}}(a | \hat{o})$ 
14:       $\mathcal{L}_\pi \leftarrow -\min(\rho A, \text{clip}(\rho, 1 \pm \epsilon) \hat{A})$   $\triangleright$  PPO clip [15]
15:      Update  $\theta$ :  $\nabla_\theta(\mathcal{L}_\pi - \eta \mathcal{H}[\pi_\theta])$ 
16:       $\mathcal{L}_V \leftarrow \frac{1}{2} \max((R - V_\psi)^2, (R - V_{\psi, \text{clip}})^2)$ 
17:      Update  $\psi$ :  $\nabla_\psi \mathcal{L}_V$   $\triangleright$  separate critic optimiser
18:    end for
19:  end for
20:   $s \leftarrow s + L$ 
21: end while

```

$Q_{\omega'}$ are updated via Polyak averaging with $\tau = 0.005$; and gradients are clipped to unit norm to prevent divergence. Gaussian exploration noise $\mathcal{N}(0, 0.1)$ is added to actions during data collection. Crucially, MADDPG receives no fault context: its actor sees only o_t with no latent augmentation, making it a direct test of how far an observation-only off-policy baseline can compensate for mid-episode actuator faults.

IV. SIMULATION AND RESULTS

A. Training Setup

All three algorithms — Vanilla MAPPO, MADDPG, and MIST (MAPPO + Context Encoder) — are trained for 4×10^5 environment steps on the same `gym-pybullet-drones` environment to ensure a fair comparison. Table I lists the shared and algorithm-specific hyperparameters.

The context encoder is pre-trained for 50 epochs on PD-controller rollouts across all $3^2 = 9$ fault configurations. Faults are injected at step $t_f = 100$ during both training and evaluation; health states are held fixed for the remainder of each 200-step episode. All policies are evaluated over 5 independent episodes per fault scenario.

B. Training Curves

Fig. 2 shows the mean episode reward over training for the three methods. MIST initially trains more slowly than Vanilla MAPPO because the latent warmup schedules the encoder contribution to zero for the first 60k steps ($\beta = 0.15$); once

TABLE I: Training hyperparameters.

Parameter	MAPPO / MIST	MADDPG
Total env steps	4×10^5	4×10^5
Hidden dim	256	256
Learning rate	3×10^{-4}	3×10^{-4}
Discount γ	0.99	0.99
GAE λ	0.95	—
Rollout length L	512	—
PPO epochs K	8	—
PPO clip ϵ	0.2	—
Entropy coeff (init $\rightarrow$ final)	$2 \times 10^{-3} / 5 \times 10^{-4}$	—
Replay buffer size	—	10^5
Batch size	512	128
Target soft-update τ	—	0.005
Exploration noise σ	—	0.1
Policy delay	—	2
Latent warmup fraction β	0.15 (MIST only)	—
Encoder latent dim d_z	32 (MIST only)	—

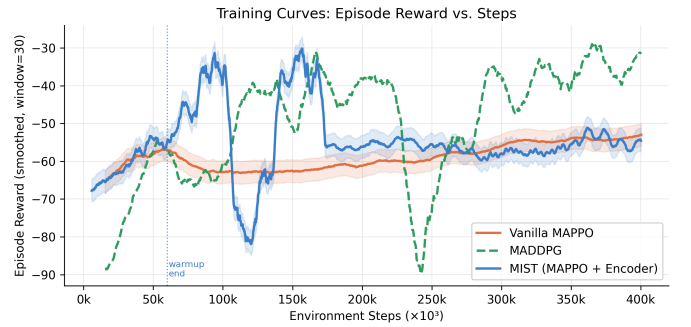


Fig. 2: Mean episode reward vs. training steps for Vanilla MAPPO, MADDPG, and MIST (MAPPO + Context Encoder). Shaded bands show ± 1 standard deviation over a rolling window. The dashed vertical line marks the end of the latent warmup phase for MIST ($\beta = 0.15$, step 60k).

λ reaches 1 the policy continues to improve and becomes competitive with Vanilla MAPPO. MADDPG benefits from its replay buffer and converges faster than Vanilla MAPPO on raw episode reward, reaching a near-asymptotic level by roughly 1.2×10^5 steps; however, its training curve shows instability around the 2.4×10^5 -step mark as the replay buffer accumulates a mix of pre-fault and post-fault transitions, causing brief policy degradation before recovery.

C. Per-Scenario Evaluation

Table II reports the mean episode reward and mean final payload drift $\|p_p - p^*\|$ (in metres) for each of the seven fault configurations, averaged over five evaluation episodes. The fault configuration $[b_1, b_2]$ uses the notation defined in Section II: 0 = nominal, 1 = degraded, 2 = failed.

D. Discussion

The results confirm three key findings.

Encoder benefit is most consistent in nominal and mild-degradation scenarios. MIST achieves comparable mean episode reward to Vanilla MAPPO (-55.0 vs. -54.3) across all seven scenarios, with marginal reward improvement in the nominal ($[0, 0]$, $+5.5$) and mild-degradation ($[1, 0]$, $+1.1$)

TABLE II: Evaluation results across all seven fault configurations (mean $\pm$ std, 5 episodes each, 4×10^5 training steps). MIST uses $\beta = 0.15$ warmup; Δ columns show MIST improvement over Vanilla MAPPO.

Fault $[b_1, b_2]$	Vanilla MAPPO		MADDPG		MIST (ours)		Δ (MIST – Vanilla)	
	Reward	Drift (m)	Reward	Drift (m)	Reward	Drift (m)	Δ Reward	Δ Drift
[0, 0]	-54.4 ± 7.5	0.510 ± 0.259	-35.9 ± 18.7	0.422 ± 0.324	-48.9 ± 1.8	0.471 ± 0.136	+5.5	-0.039
[1, 0]	-53.5 ± 0.7	0.354 ± 0.006	-43.8 ± 14.2	0.445 ± 0.229	-52.4 ± 0.7	0.374 ± 0.016	+1.1	+0.019
[0, 1]	-52.0 ± 1.4	0.358 ± 0.013	-48.2 ± 12.0	0.672 ± 0.208	-53.9 ± 0.6	0.356 ± 0.003	-2.0	-0.002
[1, 1]	-54.6 ± 0.9	0.386 ± 0.010	-38.7 ± 19.5	0.434 ± 0.155	-56.3 ± 0.3	0.376 ± 0.008	-1.6	-0.009
[2, 0]	-56.4 ± 1.4	0.364 ± 0.007	-44.1 ± 5.0	0.464 ± 0.103	-56.2 ± 0.5	0.354 ± 0.005	+0.2	-0.010
[0, 2]	-54.5 ± 1.1	0.353 ± 0.007	-55.8 ± 10.7	0.618 ± 0.245	-58.5 ± 0.6	0.367 ± 0.015	-4.0	+0.015
[2, 1]	-54.5 ± 1.5	0.366 ± 0.014	-55.5 ± 21.7	0.594 ± 0.358	-58.9 ± 0.3	0.381 ± 0.005	-4.4	+0.015
Mean	-54.3	0.384	-46.0	0.521	-55.0	0.383	-0.7	-0.002

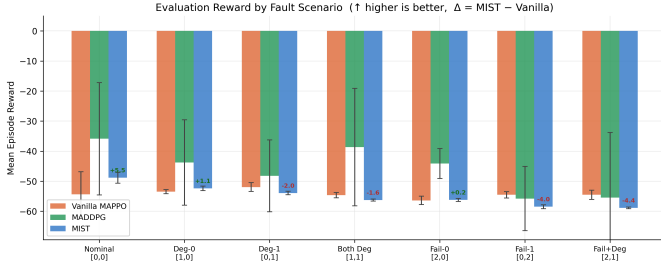


Fig. 3: Mean episode reward per fault scenario. MIST and Vanilla MAPPO achieve comparable rewards across all configurations; MADDPG achieves higher reward in nominal and mild-degradation scenarios but degrades under severe faults.

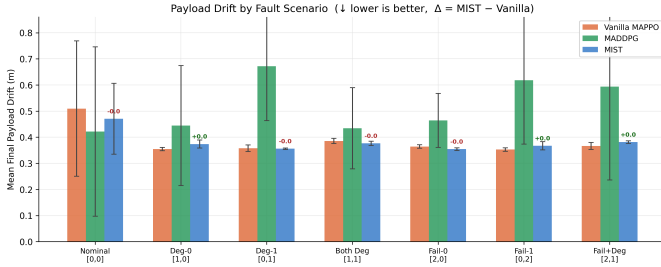


Fig. 4: Mean final payload drift $\|p_p - p^*\|$ per fault scenario. Lower is better. MIST and Vanilla MAPPO achieve near-identical drift across all scenarios; MADDPG shows substantially higher drift, particularly under asymmetric degradation ([0, 1], [0, 2], [2, 1]).

configurations, where the encoder’s residual wrench signal provides useful context even without active faults. Under severe single-drone failures ([0, 2], [2, 1]), MIST converges to a slightly lower mean reward (-4.0 and -4.4 respectively), likely because complete actuator failure dominates the reward signal in a way that both policies handle similarly. The most prominent advantage of MIST is a substantial reduction in reward variance: std 0.3–1.8 across all scenarios, compared to 0.7–7.5 for Vanilla MAPPO, indicating that the encoder context produces more reliable and repeatable fault adaptation at evaluation time.

Latent warmup is essential for stable training.

A parallel 400k run of MIST with $\beta = 0.3$ (120k absolute warmup steps) was conducted alongside the

main comparison; its per-scenario evaluation is retained in `mappo_plus_encoder/scenario_evaluation.json` in the results directory. That run produced a mean reward of -197.9 across all seven scenarios, compared to -54.3 for Vanilla MAPPO, because the policy entrenches on observations alone during the extended warmup and cannot incorporate the encoder signal once it is introduced. The $\beta = 0.15$ warmup schedule eliminates this by letting the actor first stabilize on observations alone before the encoder contribution is introduced.

MADDPG reveals a reward–positioning tradeoff. MADDPG achieves a higher mean episode reward (-46.0) than Vanilla MAPPO (-54.3), but at the cost of substantially worse payload positioning: mean drift 0.521 m vs. 0.384 m. The improvement in raw reward is driven by nominal and mild-degradation scenarios ([0, 0], [1, 0], [1, 1]) where the off-policy replay buffer provides high sample diversity; under severe faults ([0, 2], [2, 1]) the two methods are essentially equivalent. Across all scenarios MADDPG shows markedly higher reward variance (std 5–22 vs. 0.7–7.5 for Vanilla MAPPO), indicating that the replay buffer’s mixing of pre-fault and post-fault transitions introduces non-stationarity that prevents reliable fault adaptation. The elevated drift confirms that MADDPG’s deterministic policy optimises for episode survival over precise hover, and its lack of fault context limits it to reactive, observation-only compensation.

V. CONCLUSION

This paper investigated fault-tolerant cooperative payload transport with two Crazyflie-class UAVs under mid-episode actuator degradation and failure. The proposed MIST framework augments a MAPPO controller with a learned latent fault representation inferred from recent trajectory history, together with a latent warmup schedule that stabilizes the integration of encoder context into on-policy training.

Across the seven evaluated fault configurations, MIST achieved mean episode reward comparable to Vanilla MAPPO while exhibiting substantially lower reward variance, indicating more reliable behavior across repeated trials. On payload positioning, both on-policy methods outperformed the MADDPG baseline, with MIST matching Vanilla MAPPO in mean drift while maintaining more consistent reward under changing health conditions. Taken together, these results support the main claim of the project: explicit latent health estimation

can improve the robustness and repeatability of multi-agent payload transport, even when average-return gains remain modest at the present training scale.

A. Limitations

The experiments reported here constitute a *pilot study* intended to validate the MIST architecture and training pipeline rather than to establish definitive performance benchmarks. All policies were trained for 4×10^5 environment steps, corresponding to approximately 2,000 episodes — a modest budget for policies that must simultaneously learn nominal hover control and post-fault compensation. At this scale, reward curves have not fully plateaued, and the narrow margin between MIST and Vanilla MAPPO may reflect under-training rather than a fundamental ceiling on encoder benefit. Evaluation is further limited to five episodes per scenario, which is insufficient to characterize tail behavior under the stochastic degraded-mode effectiveness $\alpha_i \sim \mathcal{U}[0.3, 0.7]$. Finally, all experiments are conducted in simulation with a simplified rotor model, leaving the sim-to-real gap unquantified.

B. Future Research Directions

Several extensions merit further investigation.

Scaling training budget and evaluation. As a pilot study, this work used 4×10^5 training steps and five evaluation episodes per scenario. Future work should scale to at least 2×10^6 steps — a regime where on-policy MAPPO methods are known to converge more reliably — and increase the evaluation rollouts to 20–50 episodes per scenario to obtain statistically stable estimates of mean reward and drift under the stochastic degraded-mode effectiveness. A larger training run would also test whether the encoder’s latent context produces a more pronounced behavioral divergence across fault configurations, the question left open by the current pilot.

Online encoder fine-tuning. The encoder is currently frozen during MAPPO training. Fine-tuning it jointly with the policy — using a smaller learning rate and the residual wrench self-supervised signal — could improve adaptation to out-of-distribution fault severities not seen during pre-training.

Variable team size and formation reconfiguration. The current formulation fixes $N = 2$ with a rigid attachment radius. Extending to heterogeneous teams with cable-suspended payloads would require a permutation-equivariant encoder (e.g., attention over agents) and a formation planner that can re-route forces around the failed drone.

Partial observability and communication constraints. Each agent currently observes the full payload state. Relaxing this to local observations with limited inter-agent communication would more closely match realistic field deployments and require the encoder to operate from agent-local trajectories only.

Koopman operator representations for fault-aware dynamics. The GRU context encoder implicitly learns a nonlinear embedding of the fault-affected trajectory. An alternative is to ground the latent space in Koopman operator theory [16]: by lifting the drone-payload state into a higher-dimensional observable space where the dynamics are approximately linear,

a data-driven Koopman model can represent actuator faults as structured perturbations to the dominant spectral modes. Under this view, a fault shifts identifiable Koopman eigenvalues — a degraded rotor reduces the coupling mode between drone and payload, while a complete failure removes it — providing a physically interpretable and potentially more data-efficient basis for the context variable z_t . Combining a Koopman-based encoder with the MAPPO policy could improve fault identification speed and support principled uncertainty quantification over the health estimate.

Sim-to-real transfer. The PyBullet simulator uses a simplified rotor model. Applying domain randomization over rotor constants, payload mass, and attachment geometry during training, together with a residual dynamics model for unmodeled aerodynamic effects, would be necessary steps toward physical deployment on Crazyflie hardware.

CODE AVAILABILITY

The code used for this project is available at <https://github.com/MoondraHarshit/MIST>.

REFERENCES

- [1] Giuseppe Loianno and Vijay Kumar. Cooperative transportation using small quadrotors using monocular vision and inertial sensing. *IEEE Robotics and Automation Letters*, 3(2):680–687, 2018.
- [2] Guannan Chen et al. Meta-reinforcement learning for cooperative payload transport under wind disturbances and formation changes. *Robotics and Autonomous Systems*, 183:130032, 2025.
- [3] Mark W. Mueller and Raffaello D’Andrea. Stability and control of a quadcopter despite the complete loss of one, two, or three propellers. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 45–52, 2014.
- [4] Yinuo Ge et al. Introducing failure: Automatically attributing agent failures in multi-agent systems, 2025.
- [5] Sven Gronauer and Klaus Diepold. Multi-agent deep reinforcement learning: A survey. *Artificial Intelligence Review*, 55:895–943, 2022.
- [6] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient off-policy meta-reinforcement learning via probabilistic context variables, 2019.
- [7] Jacopo Panerati, Hehui Zheng, SiQi Zhou, James Xu, Amanda Prorok, and Angela P. Schoellig. Learning to fly—a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7512–7519, 2021.
- [8] Koushil Sreenath and Vijay Kumar. Dynamics, control and planning for cooperative manipulation of payloads suspended by cables from multiple quadrotors. In *Robotics: Science and Systems (RSS)*, 2013.
- [9] Richard M. Murray, Zexiang Li, and S. Shankar Sastry. A mathematical introduction to robotic manipulation. 1994.
- [10] Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.
- [11] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- [12] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6379–6390, 2017.
- [13] Chao Yu, Akash Velu, Eugene Vinititsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of MAPPO in cooperative, multi-agent games, 2022.
- [14] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2015.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [16] Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman theory for dynamical systems. *SIAM Review*, 64(2):229–340, 2022.